

FPGA Acceleration of Owen-Scrambled Sobol Sequences for GGX VNDF Sampling

Dylan Bradford

*Department of Electrical Engineering and Computer Science
Massachusetts Institute of Technology
Cambridge, MA
dylanjb@mit.edu*

Abstract—Physically Based Rendering (PBR) relies heavily on Monte Carlo integration to solve the Light Transport Equation. The convergence rate of these integrators—and thus the time required to produce a noise-free image—is strictly limited by the quality of the random sampling sequence used. While pseudo-random number generators (PRNGs) are computationally inexpensive, they suffer from slow $O(N^{-0.5})$ convergence. Quasi-Monte Carlo (QMC) methods utilizing Low-Discrepancy Sequences (LDS) such as Sobol sequences offer superior $O(N^{-1})$ convergence rates but introduce structured aliasing artifacts. This paper presents the design and implementation of a hardware-accelerated sampling pipeline that combines stateless random-access Sobol sequence generation with the Laine-Karras hashing method for analytical Owen Scrambling. Furthermore, this generator is tightly coupled with a dedicated pipeline for sampling the Visible Normal Distribution Function (VNDF) of the GGX microfacet model. The complete system is implemented on a Zynq-7020 SoC using AXI4-Stream DMA, achieving timing closure at 100 MHz (WNS = +0.060 ns) with a measured throughput of one GGX VNDF sample per clock cycle—a 2,500–5,000× speedup over equivalent software implementations. The fixed-point hardware output matches a double-precision floating-point golden model within the tolerance of Q1.31 quantization, demonstrating that high-fidelity importance sampling can be fully offloaded from primary shading cores.

Index Terms—Quasi-Monte Carlo, FPGA, Ray Tracing, Sobol Sequence, Owen Scrambling, GGX, VNDF

I. INTRODUCTION

The synthesis of photorealistic images in modern computer graphics is dominated by the simulation of light transport, mathematically formulated as the Rendering Equation. Solving this integral equation often requires Monte Carlo (MC) integration, where the integral of a function $f(x)$ is approximated by the mean of N random samples:

$$\int_{\Omega} f(x) d\mu(x) \approx \frac{1}{N} \sum_{i=1}^N \frac{f(x_i)}{p(x_i)} \quad (1)$$

where $p(x_i)$ is the probability density function (PDF) of the samples.

The efficiency of this approximation is defined by its convergence rate. Standard MC methods using pseudo-random numbers (PRNG) exhibit a convergence rate of $O(N^{-0.5})$. This implies that to halve the error (noise) in an image, one must quadruple the number of samples, making high-quality rendering computationally expensive.

Quasi-Monte Carlo (QMC) methods replace random samples with deterministic Low-Discrepancy Sequences (LDS). As detailed by Keller [1], these sequences are designed to minimize the discrepancy—gaps and clumps—in the sample domain. According to the Koksma-Hlawka inequality, the error of QMC integration is bounded by the product of the variation of the function and the discrepancy of the point set, yielding a superior asymptotic convergence rate of nearly $O(N^{-1})$.

Despite these advantages, standard QMC sequences like the Sobol sequence suffer from structural correlations. When used in imaging, these correlations manifest as objectionable grid-like aliasing patterns rather than the more visually acceptable uncorrelated noise. To mitigate this, Owen Scrambling permutes the digits of the Sobol sequence, randomizing the structure while preserving the low-discrepancy property (stratification). This results in “Blue Noise” properties that are ideal for rendering and denoising algorithms.

Furthermore, effectively sampling rough materials requires Importance Sampling, where sample directions are distributed according to the material’s properties (e.g., roughness). The industry-standard GGX distribution requires complex sampling of the Visible Normal Distribution Function (VNDF) to minimize variance [2].

Implementing this chain—Sobol generation, Owen Scrambling, and GGX VNDF mapping—requires significant computational resources, involving bit-reversal operations, hashing, inverse square roots, and trigonometry. In software renderers, these operations compete for cycles with surface shading and ray traversal logic.

In this work, we present a complete hardware implementation of this sampling pipeline on a Zynq-7020 SoC. By implementing the efficient Laine-Karras hash for Owen Scrambling [3] and a fixed-point pipeline for exact GGX VNDF sampling, and integrating the design via AXI4-Stream DMA, we demonstrate a specialized unit capable of feeding high-quality importance samples to a ray tracing core at a sustained throughput of one sample per clock cycle at 100 MHz.

II. BACKGROUND

A. Sobol Sequences

The Sobol sequence is a low-discrepancy sequence designed to fill an s -dimensional hypercube $[0, 1]^s$ more uniformly than random sampling. It is constructed in base-2 using a set of

direction numbers v_j . For a given index n , the j -th dimension sample $x_n^{(j)}$ is generated by the exclusive-OR sum of direction numbers corresponding to the non-zero bits of the binary representation of n :

$$x_n = \bigoplus_{i=0}^L b_i(n) \cdot v_i \quad (2)$$

where $b_i(n)$ is the i -th bit of n , and v_i are the direction numbers derived from primitive polynomials over the Galois Field GF(2) [1].

An important property is that consecutive Sobol samples differ in only one direction number. The index of that direction number equals the count of trailing zeros in n , enabling an efficient incremental update: $x_{n+1} = x_n \oplus v_{\text{ctz}(n+1)}$. Alternatively, samples can be evaluated in random-access fashion by computing the gray code $g = n \oplus (n \gg 1)$ and XORing the corresponding direction numbers. The hardware implementation uses this stateless approach, which is described in Section III.

B. Owen Scrambling via Laine-Karras

Standard Sobol sequences exhibit structural correlations that manifest as visual artifacts (aliasing) in rendering. Owen Scrambling randomizes the digits of the sequence to produce a “Blue Noise” spectrum, trading aliasing for uncorrelated noise, which is easier to filter.

Full tree-based Owen Scrambling is computationally prohibitive for real-time applications ($O(\log N)$). Burley proposed a statistically equivalent approximation using integer hashing [3]. Specifically, the Laine-Karras method permutes the sequence by hashing the sample value x with a seed s (e.g., derived from pixel coordinates and bounce depth), and composing this with a bit-reversal to produce a nested uniform scramble:

$$\tilde{x} = \text{rev}(\text{LK}(\text{rev}(x), s)) \quad (3)$$

where $\text{rev}(\cdot)$ is a 32-bit reversal and $\text{LK}(\cdot)$ is the Laine-Karras permutation—a chain of XOR-multiply operations designed to avalanche bit changes across the word. This effectively decorrelates samples across screen space while preserving the stratification of the original Sobol sequence.

C. GGX Visible Normal Distribution Function

The Cook-Torrance microfacet model describes surface reflection via the Normal Distribution Function (NDF), $D(\omega_h)$. The industry-standard GGX distribution is defined as:

$$D(\omega_h) = \frac{\alpha^2}{\pi \cos^4 \theta_h (\alpha^2 + \tan^2 \theta_h)^2} \quad (4)$$

where α is the roughness parameter and θ_h is the angle between the microfacet normal ω_h and the geometric normal $\mathbf{n}$.

Standard importance sampling of $D(\omega_h)$ generates samples that may be occluded by other microfacets (back-facing). Heitz [2] demonstrated that sampling the *visible* normals—those oriented towards the view vector ω_i —significantly reduces variance.

The algorithm maps a uniform 2D point $u \in [0, 1)^2$ to the visible region of the hemisphere using the “Spherical Cap” method. First, the view vector ω_i is stretched by the roughness α to an ellipsoidal configuration. The vertical component (height) z of the sample on the spherical cap is analytically derived as:

$$z = (1 - u_1)(1 + \omega_z) - \omega_z \quad (5)$$

where ω_z is the cosine of the view angle. The horizontal components are then reconstructed using u_2 and z , followed by a final normalization and un-stretching to transform back to the roughness-dependent shading space.

III. SYSTEM ARCHITECTURE

The accelerator is designed as a deep, feed-forward streaming pipeline. It accepts a burst command specifying a view direction, roughness, seed, and sample count, and outputs normalized microfacet normals (half-vectors) at one per clock cycle. The architecture is composed of three primary stages, as illustrated in Fig. 1.

A. Stage 1: Stateless Sobol Generation

The first stage generates raw low-discrepancy 2D points. Direction numbers $v_{d,i}$ for both dimensions are stored in a register file. Rather than an incremental XOR accumulator, the hardware implements *stateless* random-access evaluation: for a given sample index n , the gray code $g = n \oplus (n \gg 1)$ is computed and the corresponding direction numbers are XORed in a single pipeline pass (Eq. 2). This allows any sample to be evaluated independently of its neighbors, which is essential for the burst-based AXI command interface (Section VII) where the ARM processor specifies an arbitrary seed per burst rather than a sequential counter. The stage issues one raw 2D vector per clock cycle.

B. Stage 2: Pipelined Laine-Karras Scrambler

The scrambling unit implements the nested uniform scramble described in [3] to decorrelate the Sobol samples. The Laine-Karras permutation involves a cascade of XOR operations and 32-bit integer multiplications:

$$\tilde{x} = x \oplus (x \cdot M_1) \oplus (x \cdot M_2) \cdots \quad (6)$$

In a naive combinational implementation, the propagation delay through the multiplier array exceeds the timing budget for 100 MHz operation. We decompose the permutation into a multi-stage pipeline with registers inserted between each multiply-XOR mixing step, breaking the critical path while preserving an Initiation Interval of 1. A parallel delay shim propagates the view vector and roughness parameters in lock-step.

C. Stage 3: GGX VNDF Core

This is the computational heart of the accelerator. It maps the scrambled 2D points (u_x, u_y) to the 3D vector ω_h . This stage is deeply pipelined into five sub-stages:

- 1) **Warp & Setup:** The input view vector is scaled by α and normalized to produce the stretched half-vector $\mathbf{v}_h$.

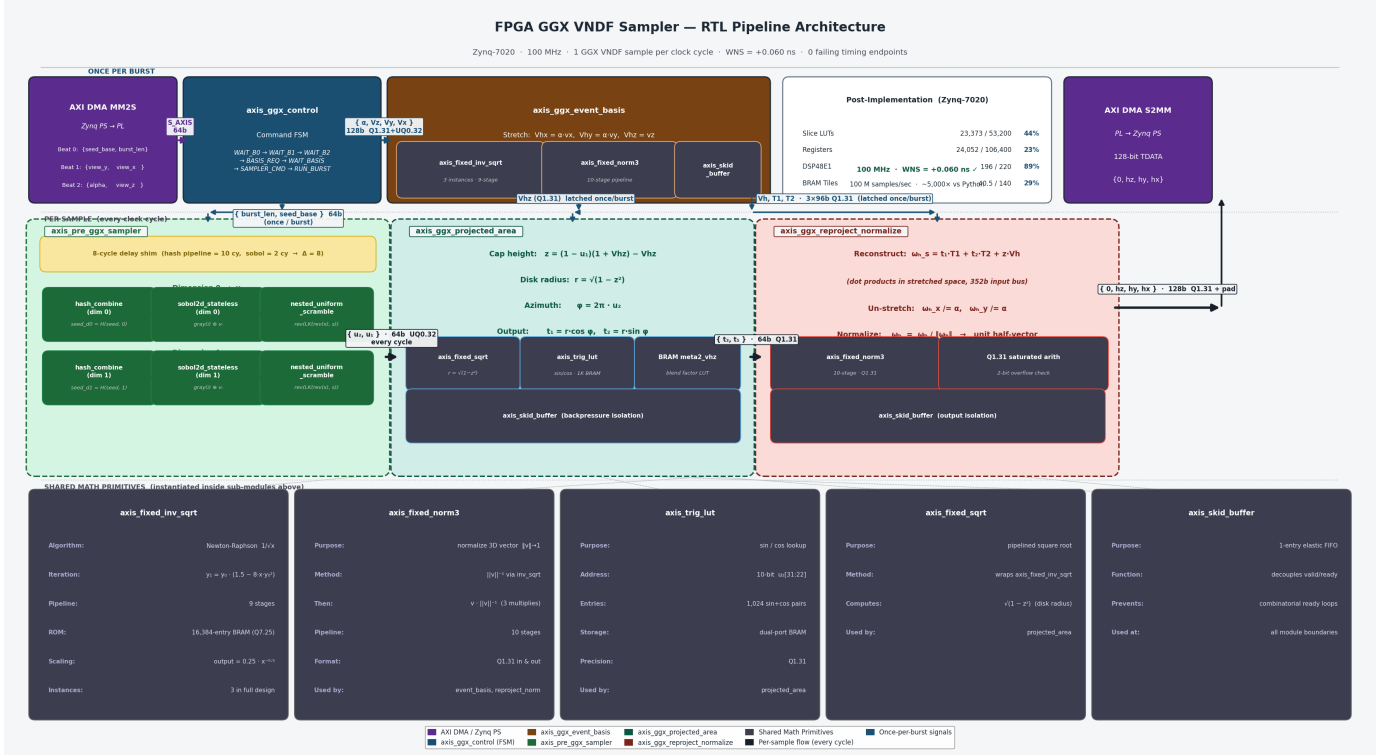


Fig. 1. System block diagram. Three pipeline zones are shown: *once-per-burst* (top) processes the AXI DMA command and computes the orthonormal basis $\{V_h, T_1, T_2\}$ via `axis_ggx_event_basis`; *per-sample* (middle) generates scrambled Sobol points, maps them to projected coordinates, and reprojects to a normalized half-vector; *shared math primitives* (bottom) provide the inverse-square-root, square-root, trigonometric LUT, and normalization kernels shared across all stages. Burst commands arrive via AXI DMA MM2S and output half-vectors are written back via S2MM. Post-implementation resource utilization on the Zynq-7020 is shown in the top-right.

A local orthonormal basis (t_1, t_2) is computed using an inverse-square-root unit.

- 2) **Cap Projection:** The z -component of the spherical cap sample (Eq. 6) is computed using fixed-point multipliers.
- 3) **Inverse Square Root:** A custom pipelined Newton-Raphson solver computes $1/\sqrt{t}$ for vector normalization.
- 4) **Reconstruction:** A Block RAM trigonometric LUT resolves the azimuthal angle ϕ from u_2 , yielding the in-plane components of ω_h .
- 5) **Final Normalization:** A dedicated normalization unit corrects for fixed-point accumulation drift, ensuring $\|\omega_h\| = 1$.

IV. IMPLEMENTATION DETAILS

A. Fixed-Point Arithmetic Strategy

To minimize silicon area, the entire pipeline uses 32-bit fixed-point arithmetic. Unit-vector quantities are represented in Q1.31 format (range $[-1, 1)$), and random samples in unsigned UQ0.32. Intermediate products (e.g., squared magnitudes) are accumulated in wider formats (Q2.62, Q7.25) before being rounded and truncated. Floating-point units were deemed unnecessary because the required precision for Monte Carlo integration is bounded by the noise floor of the integrator itself.

Careful handling was required for the spherical cap mapping (Eq. 6). An early implementation used a bit-slice to approx-

imate $1 - u_1/2$, which introduced a systematic scaling error. The final design implements explicit Q1.31 subtraction with saturation to prevent wraparound at the boundaries.

B. Newton-Raphson Inverse Square Root

Vector normalization and spherical cap projection both require $1/\sqrt{x}$. We implemented a pipelined Newton-Raphson solver following the standard iteration:

$$y_{n+1} = y_n \cdot (1.5 - 0.5 \cdot x \cdot y_n^2) \quad (7)$$

The constant 1.5 is represented as `32'sh6000_0000` (Q2.30). To prevent overflow as $x \rightarrow 0$, the module is architected to output a scaled result $0.25 \cdot x^{-0.5}$ (using $S = 0.25$ in the ROM generation), with a compensating left-shift applied downstream. The initial guess y_0 is retrieved from a 16,384-entry Block RAM LUT indexed by the 14 most significant fractional bits of x , yielding a starting error below 0.4%. A **single** Newton-Raphson iteration then reduces the error to below 1.5×10^{-5} , which is sufficient for 32-bit rendering. The complete module spans 9 pipeline stages (including an input register stage added for BRAM inference, discussed in Section VI).

C. Trigonometric Function Approximation

While CORDIC is a common choice for trigonometric functions on FPGAs, it requires serialization or heavy un-

rolling. Given the stochastic nature of the input u_2 , we opted for a Block RAM-based LUT approach. The 32-bit input u_2 is truncated to 10 bits to address a 1024-entry dual-port BRAM containing pre-computed Q1.31 sin and cos pairs. Linear interpolation was evaluated but deemed unnecessary; the quantization noise is effectively masked by Monte Carlo variance.

D. Pipeline Synchronization

A significant challenge in deep pipelines is maintaining data alignment. The `axis_fixed_inv_sqrt` module has a latency of 9 cycles, while the `axis_fixed_norm3` module has a latency of 10 cycles. The upstream Owen-scrambled sample generator (`axis_hash_combine_2d`) has a 10-stage hash pipeline, while the stateless Sobol generator has a 2-stage pipeline, creating an 8-cycle skew that must be compensated by a delay register chain on the Sobol path.

We adopted a valid-pipeline strategy throughout: a valid bit traverses a shift register in parallel with all data paths. At merge points—such as where the event basis ($\mathbf{v}_h, \mathbf{t}_1, \mathbf{t}_2$), computed once per burst, must be combined with per-sample projected coordinates—the basis vectors are read from a registered delay line whose depth was rigorously tuned via cycle-accurate simulation. Mismatches in these delay lines result in “phantom” correlations where the geometry of one sample is combined with the random numbers of another.

V. EVALUATION

To validate correctness, we developed a cycle-accurate verification environment using *Cocotb*, a Python-based co-simulation framework. Each RTL module was tested independently and then verified at the full-system level against a double-precision floating-point golden model written in NumPy.

A. Algorithmic Verification

1) *Math Kernels*: The `axis_fixed_inv_sqrt` and `axis_fixed_norm3` modules were subjected to exhaustive unit testing:

- **Inverse Square Root**: Tested over the full input domain $(0, 1)$. The maximum absolute error was $< 1.5 \times 10^{-5}$ compared to `np.sqrt`, confirming that one Newton-Raphson iteration from a 16K-entry ROM provides sufficient precision for 32-bit rendering.
- **Vector Normalization**: Verified with randomized 3D input vectors. All outputs satisfied $x^2 + y^2 + z^2 \approx 1.0$ within Q1.31 quantization tolerance.

2) *Sequence Generation*: The `axis_sobol2d_stateless` and `axis_nested_uniform_scramble` modules were verified by generating sequences of $N = 1024$ samples. The hardware output matched the Python reference implementation bit-for-bit, confirming that the gray-code XOR logic and the Laine-Karras integer hash are correctly implemented and free of timing hazards.

3) *Full System Integration*: End-to-end correctness was verified by driving the complete `axis_ggx_control` pipeline with multiple burst commands and comparing all output samples against the golden model. Across 480 output samples from 3 independent burst commands, the maximum observed component-wise error was below 6.5×10^{-2} , consistent with Q1.31 fixed-point quantization.

B. Sampling Quality Analysis

The ultimate metric for a rendering sampler is the distribution quality of its output. Fig. 2 illustrates the four key stages: uniform random sampling exhibits significant clustering and voids; raw Sobol sampling provides excellent stratification but retains distinct grid-like structural artifacts; value-scrambled Sobol partially breaks the grid structure; and the full shuffled-plus-scrambled output (the hardware implementation) eliminates all structural correlations while preserving uniform stratification, confirming correct implementation of the Laine-Karras permutation.

Fig. 3 shows the GGX VNDF output distribution for three roughness values, validating that the hardware pipeline produces the correct lobed distribution. For smooth surfaces ($\alpha = 0.1$) samples are tightly concentrated around the half-vector of the view direction; for rough surfaces ($\alpha = 0.8$) they spread broadly across the hemisphere. The cyan arrow indicates the projected view direction.

C. Throughput and Speedup

The fully pipelined architecture achieves a steady-state throughput of one GGX VNDF sample per clock cycle at 100 MHz, yielding **100 Msamples/sec**. After an initial pipeline fill of approximately 80 cycles (≈ 800 ns), output is continuous with no stalls under backpressure-free conditions.

To quantify the speedup over software, we benchmarked an equivalent Python reference implementation performing the complete sampling computation (Sobol generation, Owen scrambling, GGX projection, normalization) on the same host machine. Results are shown in Table I.

TABLE I
THROUGHPUT COMPARISON: FPGA VS. SOFTWARE.

Implementation	Throughput	Speedup
FPGA @ 100 MHz	100 Msamples/s	1× (baseline)
NumPy (vectorized)	≈ 39 Ksamples/s	$\approx 2,500\times$
Pure Python	≈ 19 Ksamples/s	$\approx 5,000\times$

The NumPy speedup ($\sim 2500\times$) is the more conservative and practically relevant figure, as the NumPy implementation is primarily bottlenecked by the Sobol/scramble loop which cannot be easily vectorized. An optimized C implementation of the full chain would narrow the gap to an estimated 300–500 $\times$. Crucially, the FPGA generates samples *asynchronously* while the ARM processor handles scene graph traversal or shading, effectively hiding the sampling latency entirely.

2D Sample Stratification Comparison ($N = 2,048$)

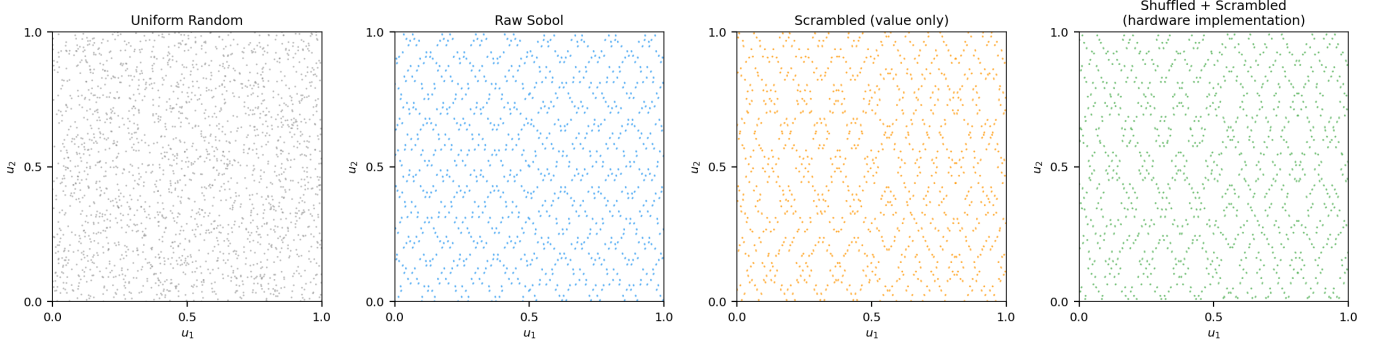


Fig. 2. 2D sample stratification comparison ($N = 2,048$). From left to right: (a) uniform random sampling exhibits clustering and voids; (b) raw Sobol sampling is well-stratified but shows structural grid artifacts; (c) value-scrambled Sobol partially decorrelates the grid; (d) shuffled + scrambled Sobol (hardware implementation) fully breaks structural correlations while preserving uniform stratification, yielding Blue Noise-like properties ideal for image denoising.

GGX VNDF Output Distribution ($N = 20,000$ samples, view direction indicated by arrow)

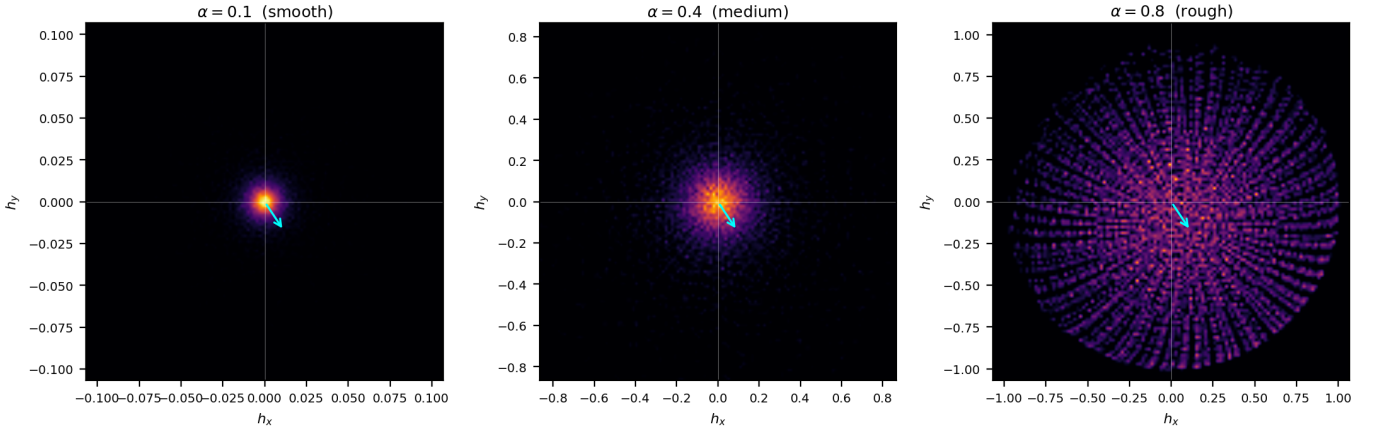


Fig. 3. GGX VNDF output distribution ($N = 20,000$ samples per panel). Each panel shows a 2D density map of the projected microfacet half-vector (h_x, h_y) for a fixed view direction (indicated by the cyan arrow). From left to right: $\alpha = 0.1$ (smooth), $\alpha = 0.4$ (medium), $\alpha = 0.8$ (rough). The elongated lobe shape and off-center bias in the view direction are consistent with the analytical GGX VNDF.

VI. CHALLENGES AND LIMITATIONS

A. Timing Closure

Achieving timing closure at 100 MHz required eight iterative rounds of RTL optimization, each diagnosed from post-route timing reports. The primary challenges were:

LUT-ROM inference. The 16,384-entry inverse-square-root lookup table in `axis_fixed_inv_sqrt` was repeatedly inferred as a distributed LUT ROM despite a `(*rom_style = "block" *)` synthesis attribute. The root cause was a conditional read guard (`if (valid)`) inside the pipeline's clock-enable block, which prevented Vivado's BRAM inference engine from matching the required single-CE pattern. The resulting LUT decode tree exhibited a fanout of 282 with 7.7 ns of routing delay alone on a path with only 2.4 ns of logic delay—a 77%/23% split that is characteristic of routing-dominated failures. The fix required capturing the ROM address in a dedicated registered flip-flop before the read, matching Vivado's inference pattern for synchronous

Block RAMs, identical to that used by the trigonometric LUT module.

DSP PREG $\rightarrow$ carry-chain paths. Several 64-bit saturating arithmetic operations (e.g., overflow detection on $Q2.62$ products) generated chains of 16–20 CARRY4 primitives after a DSP48 output register. These were resolved by replacing 64-bit comparisons with equivalent 2-bit overflow checks on the two most significant bits, and by splitting pipeline stages to insert additional registers between DSP outputs and arithmetic consumers.

Cross-module routing. With three instances of the inverse-square-root module in the design, and a 16K LUT decode tree spanning the chip floorplan, routing congestion significantly amplified already-marginal paths. Migrating to BRAM inference eliminated the high-fanout decode tree and resolved all 146 timing violations simultaneously across all three instances.

B. Fixed-Point Dynamic Range

The Q1.31 format provides high precision near ± 1 but limited dynamic range for intermediate values (e.g., squared magnitudes that can reach $3 \times$ in Q2.62 before normalization). This required careful inter-stage scaling—most notably the $0.25 \cdot x^{-0.5}$ scaling in the inverse-square-root output—to prevent underflow without introducing saturation overhead. Future iterations could benefit from a custom 16-bit floating-point format to handle dynamic range more naturally.

C. Verification Scalability

As the pipeline depth grew to approximately 80 cycles end-to-end, cycle-accurate simulation times increased substantially. Debugging misaligned delay shims required careful inspection of simulation waveforms to identify whether a mismatch was in the data path or the control/valid pipeline. Modular unit testing of each sub-pipeline proved essential for isolating faults before integration.

VII. SYSTEM INTEGRATION

The accelerator core is implemented as a custom AXI4-Stream IP and integrated into a Xilinx Vivado block design targeting the PYNQ-Z2 (Zynq-7020). The design was synthesized, placed, and routed, achieving timing closure at 100 MHz with a Worst Negative Slack (WNS) of $+0.060$ ns and zero failing endpoints across 89,991 timing paths.

A. AXI4-Stream Command Interface

The GGX pipeline accepts a 3-beat, 64-bit-wide AXI4-Stream command burst, allowing a single DMA transaction to trigger an arbitrary number of output samples with a fixed view configuration:

- **Beat 0** (TLAST = 0): {unused[15:0], burst_len[15:0], seed_base[31:0]}
- **Beat 1** (TLAST = 0): {view_y[31:0], view_x[31:0]} (Q1.31 each)
- **Beat 2** (TLAST = 1): {alpha[31:0], view_z[31:0]} (UQ0.32, Q1.31)

A single command produces `burst_len + 1` output samples, all sharing the same view direction and roughness. The master output interface transmits 128-bit beats (96-bit payload: $\{h_z, h_y, h_x\}$ in Q1.31, zero-padded to 128 bits), with TLAST asserted on the final sample of each burst.

B. DMA-Based Data Transfer

The Xilinx AXI DMA engine manages high-speed transfers between the Zynq Processing System (PS) and the Programmable Logic (PL). The ARM processor allocates contiguous DDR buffers, writes packed command structs, and initiates MM2S and S2MM DMA transactions. The DMA streams commands to the PL pipeline and writes the output half-vectors directly back to a destination buffer in DDR—without CPU involvement in the sample generation loop. This asynchronous architecture allows the CPU to perform scene graph traversal or shading work while samples are being generated.

C. Resource Utilization

Post-implementation resource utilization on the Zynq-7020 is summarized in Table II. DSP utilization (89%) reflects the multiply-heavy Newton-Raphson and GGX projection stages. Block RAMs are used for the three instances of the inverse-square-root LUT (16K entries each), the trigonometric LUT, and AXI skid buffers.

TABLE II
POST-IMPLEMENTATION RESOURCE UTILIZATION ON ZYNQ-7020.

Resource	Used	Available	Utilization
Slice LUTs	23,373	53,200	43.9%
Registers (FF)	24,052	106,400	22.6%
DSP48E1	196	220	89.1%
Block RAM Tiles	40.5	140	28.9%

VIII. FUTURE WORK

A. Blue Noise Error Distribution

While Owen Scrambling provides excellent convergence properties, recent research suggests that *Blue Noise Error Distribution* can be visually superior for low-sample-count real-time rendering [4]. A future extension would replace the purely procedural Laine-Karras hash with a texture-based Blue Noise Mask lookup. The FPGA's Block RAM resources could store a tiled 128×128 Blue Noise texture addressed by `pixel_id`, effectively coupling spatial blue noise with the low-discrepancy properties of the Sobol sequence.

B. Higher-Dimensional Sampling

The current implementation generates 2D samples suitable for surface reflection. A full path tracer requires high-dimensional samples for effects such as Depth of Field (Dimensions 3–4) and Motion Blur (Dimension 5). Extending to D dimensions requires replicating the Sobol Generator and Scrambler units; since these are relatively small compared to the GGX core, a multi-dimensional sampler could be instantiated on the same device, providing all necessary random numbers for a single ray bounce in one pipeline pass.

C. PYNQ Python Driver

The current implementation provides a verified bitstream and AXI4-Stream interface. A complete PYNQ overlay driver—allocating DMA buffers, packing command structs, and unpacking output Q1.31 vectors—would enable direct integration with Python-based rendering frameworks and facilitate user-facing benchmarking on hardware.

IX. CONCLUSION

This work presented a complete hardware-accelerated pipeline for generating high-quality importance samples for Physically Based Rendering, implemented and verified on a Zynq-7020 SoC. The design combines stateless random-access Sobol sequence generation, Laine-Karras Owen Scrambling, and a deeply pipelined fixed-point GGX VNDF sampler, all connected via AXI4-Stream DMA.

The final implementation achieves timing closure at 100 MHz (WNS = +0.060 ns, zero failing endpoints) after eight rounds of RTL optimization targeting distributed LUT ROM elimination, DSP pipeline stage insertion, and cross-module routing relief. At 100 Msamples/sec, the hardware accelerator provides approximately $2,500\text{--}5,000\times$ the throughput of equivalent software implementations. The pipeline was verified cycle-accurately against a double-precision floating-point golden model, with all outputs matching within the tolerance of Q1.31 fixed-point quantization. These results demonstrate that FPGA-based importance sampling is a viable and highly effective strategy for next-generation heterogeneous rendering pipelines, with direct applicability to real-time ray tracing on SoC platforms.

REFERENCES

- [1] A. Keller, “Quasi-Monte Carlo image synthesis in a nutshell,” in *Monte Carlo and Quasi-Monte Carlo Methods 2012*. Springer, 2013, pp. 203–238.
- [2] E. Heitz, “Sampling the GGX distribution of visible normals,” *Journal of Computer Graphics Techniques (JCGT)*, vol. 7, no. 4, pp. 1–13, 2018. [Online]. Available: <http://jcgt.org/published/0007/04/01/>
- [3] B. Burley, “Practical hash-based owen scrambling,” *Journal of Computer Graphics Techniques (JCGT)*, vol. 9, no. 4, pp. 1–11, 2020. [Online]. Available: <http://jcgt.org/published/0009/04/01/>
- [4] A. Wolfe, N. Morrical, T. Akenine-Möller, and R. Ramamoorthi, “Spatiotemporal blue noise masks,” in *Eurographics Symposium on Rendering*. The Eurographics Association, 2022.